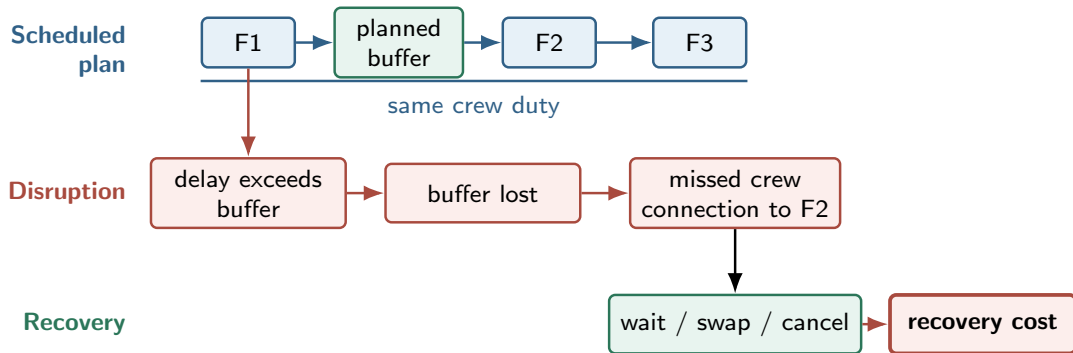


Introduction: delays turn cheap crew schedules into costly ones



Motivation: A schedule that is cheap on paper may become expensive when a delay breaks crew connections.

Incremental Data-Driven Robust Crew Pairing Optimization

Under Evolving Delay Uncertainty

Xinyu HUANG

Department of Industrial and Systems Engineering
The Hong Kong Polytechnic University

Research question: How can airline crew schedules stay reliable when delay patterns keep changing?

Introduction: this motivates the research question

Key terms

- **Crew pairing:** a sequence of flights assigned to one crew duty plan.
- **Delay propagation:** one delayed flight breaks a later crew connection.
- **Robust schedule:** a plan designed to remain workable under severe but plausible delays.

Why this matters

Planning should consider recovery cost, not only planned operating cost.

Research question

How can we design crew pairings that balance planned operating cost and recovery cost when delay patterns keep changing?

Presentation roadmap

- Literature review: what tools already exist?
- Methods: how is the model built and solved?
- Results and discussion: what can be concluded now?

Literature review: useful tools are available, but not combined

Crew pairing models

Build schedules that satisfy duty, rest, and connection rules.

Two-stage robust optimization

Separate pre-delay planning from post-delay recovery. [Zeng and Zhao, 2013](#)

Robust optimization

Control conservatism with an uncertainty budget. [Bertsimas and Sim, 2004](#)

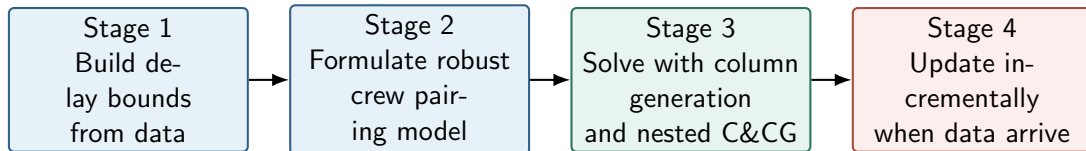
Delay-robust transport models

Use one-sided bounds for late-arrival uncertainty. [Wang and Qi, 2020](#)

Research gap

Gap: no unified framework combining data-driven delay bounds, recovery decisions, and incremental re-optimization for robust crew pairing.

Methods and procedures: the proposed framework has four stages



The key idea

The framework searches for crew schedules that are affordable in advance and easier to recover after severe delays.

Stage 1: delay data become delay bounds

Input to output

Historical delay data are converted into one-sided delay bounds.

Numerical example

Historical delays (min)

5	8	12	13	18
20	25	31	40	52

90% quantile



Delay bound = 40 min

Formula

$$d_f = \text{Quantile}_\alpha(x_f^1, x_f^2, \dots, x_f^T)$$

$$0 \leq \Delta_f \leq d_f$$

Interpretation

The model prepares for severe but realistic late arrivals.

Stage 2: the model balances planned operating cost and recovery cost

Core optimization objective

$$\min_x \{\text{planned operating cost} + \text{worst-case recovery cost}\}$$

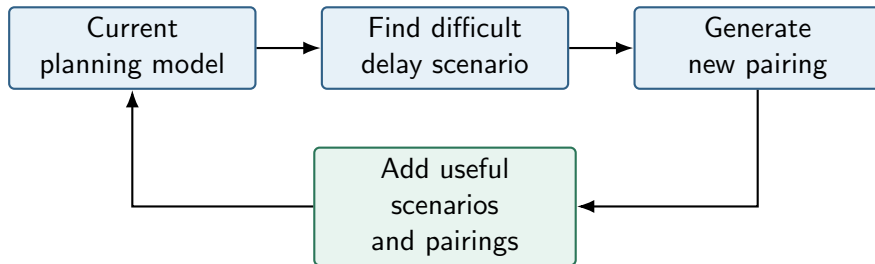
Trade-off

A slightly higher planned operating cost may reduce worst-case recovery cost.

Solved iteratively

The algorithm adds only useful pairings and difficult delay scenarios.

Stage 3: the solution adds only useful pieces



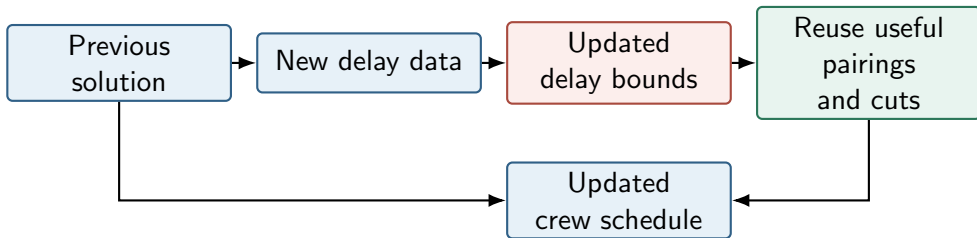
Column generation

Generate a new crew pairing only when it can improve the plan.

Nested C&CG

Generate a difficult delay scenario only when the current plan fails under it.

Stage 4: new data trigger an incremental update



Technical term

An incremental update reuses the previous solution after new data arrive.

Why this matters

Fast updates matter because delay patterns change over time.

Results and discussion: preliminary findings support the approach

Finding 1

Search becomes more targeted as useful pairings and scenarios are added.

Finding 2

Existing pairings and cuts can be reused after delay updates.

Finding 3

Higher planned cost may reduce worst-case recovery cost.

Current status

- Preliminary analytical findings
- Numerical experiments ongoing

Expected computational evaluation

Nominal model

Low planned cost, but no explicit delay protection.

Static robust model

Solves once with a fixed uncertainty set for delay protection.

Incremental robust model

Updates after new delay data while reusing previous work.

Metrics

- Planned operating cost
- Worst-case recovery cost
- Total cost
- Runtime

Interpretation

Test whether incremental re-optimization saves time while maintaining recovery performance.

Discussion: the comparison should reveal cost, reliability, and speed

Expected pattern

- Nominal plans may look cheapest before delays.
- Static robustness should reduce recovery cost.
- Incremental robustness should update faster.

Important qualification

Expected outcomes; experiments still needed.

Why this matters

The goal is a schedule that is affordable, reliable, and quick to update.

Conclusion: delay data can guide robust crew schedules

Problem and method

- Delay propagation creates recovery cost.
- One-sided delay bounds represent late arrivals.
- Robust crew pairing balances planned and recovery cost.

Contribution and next step

- Incremental updates reuse previous work.
- Faster re-optimization supports changing data.
- Numerical experiments will test the benefit.

Takeaway

Reliable schedules without excessive re-optimization time.

Thank you

Questions are welcome

Possible discussion points

Delay data, recovery actions, model comparison, or future numerical experiments.